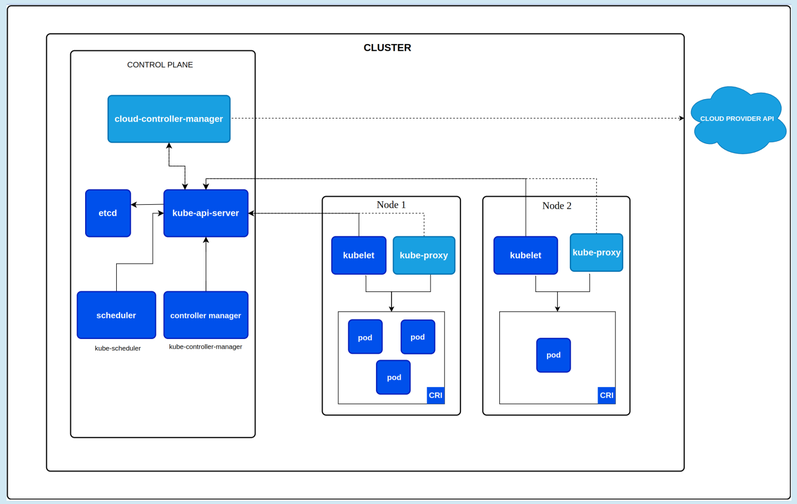
Decentralising Kubernetes: Edge Self-healing

Davide Lamagna
davide.lamagna@estudiantat.upc.edu

Kubernetes follows a master-slave architecture and is designed to be centralised for the core purpose of maintaining consistency throughout the cluster and for ease of management. This study explores possible use cases of introducing partial decentralisation to overcome limitations of the status quo architecture.



01. Introduction

Today's Kubernetes edge deployments still depend on a live control-plane to reschedule lost Pods. Whilst architectures such as KubeEdge and KubeFed allow the separation of edge nodes from the central control plane, a connection to an API server is still required for access to a scheduler.
When the WAN link fails, edge devices common in IoT deployments, such as edge cameras, factory gateways, or even retail PoS servers run a risk of not being able to restart, which may result in data gaps, service outages, or even safety alarms that never fire.
Goal: give every edge node just enough of a "brain" to detect failure, agree who should host the replacement, and launch it locally, without intervention from the central control plane.

02. Status quo architecture

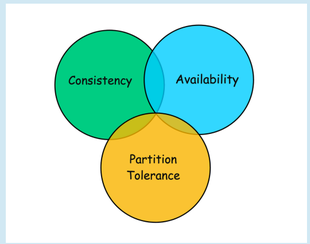
Kubernetes is designed to be centralised and have a tightly-coupled brain that gives the rest of the cluster a single, authoritative view of desired state. The control plane contains the below 4 components:

- kube-controller-manager: runs controllers which monitor the system and drive the actual state of the system towards desired state (e.g. replicas)
- etcd: the database storing the state of every object accepted by the api server, designed to be strongly consistent and built on a Raft consensus of a 3-7 node quorum.
- kube-api-server: validates, transforms, and stores every REST request. Acts as a single front door to prevent update races and guarantee the order of writes to etcd.
- kube-scheduler: watches fro unscheduled pods, scores all nodes, and issues a "bind" decision (allocating a pod to a specific node)

03. Limitations of the status quo

For the regular cluster-user, the centralised control plane makes sense. Upon investigation, there are two potential use cases where decentralisation may help.

1. Hyper mesh clusters. The rigid etcd and api-server eventually act as a bottleneck for clusters of large dimensions.
2. IoT networks. Moments of partition from the control plane result in a "freeze-brain" moment.



It comes down to basic principles - Kubernetes favours availability by design.

What if we shifted the balance to prioritise partition tolerance?

04. Prototype proposal: kubernetes-edge-healer

A lightweight self-healing DaemonSet agent (controller) for Kubernetes edge clusters. Whenever the control-plane (API server) becomes unavailable, the agent gossips liveness & free-capacity among peers, caches desired ReplicaSet specs locally, and optimistically re-binds missing Pods on the healthiest survivor node. When the WAN link returns, it reconciles duplicates to prevent split-brain.

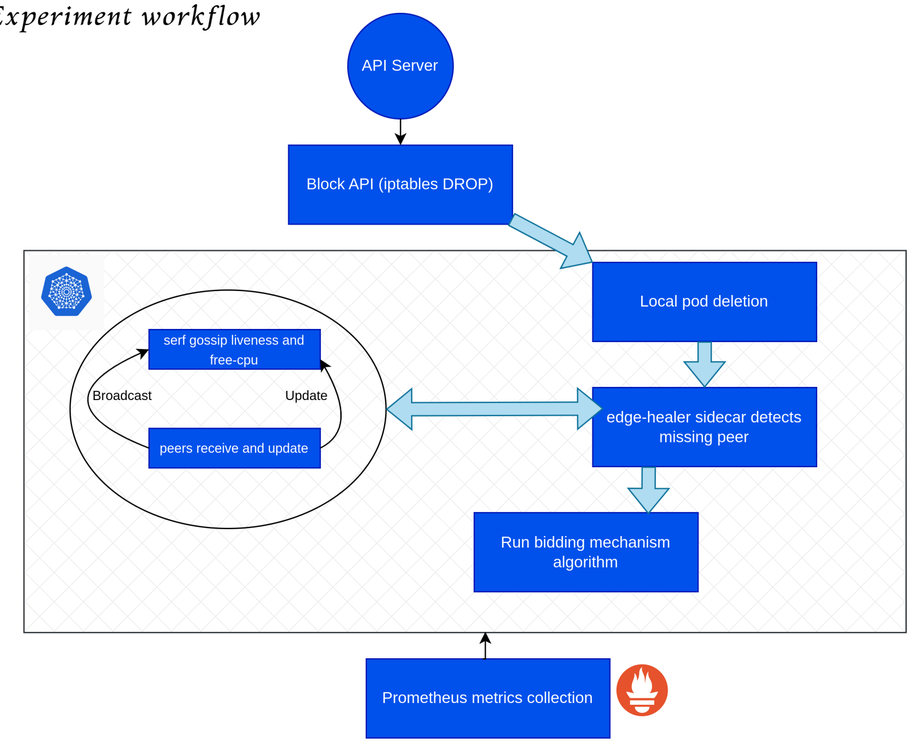
Technologies used:

- The scheduler was written in Python and containerised on each node for prototype.
- Go is the native language of Kubernetes.
- A KinD (Kubernetes in Docker) of a control plane and 3 VM nodes
- Python packages: kubernetes, kubernetes-asyncio, kopf, kubernetes-serf
- Scheduler and gossip code containerised as a side car in each worker node.
- Previous serf implementation (2015): https://github.com/slack/kubernetes-serf

Results:

- Reduced pod restore latency (2 seconds)
- Low gossip bandwidth on the 3 VM ring
- Reduced bandwidth usage between control plane and the worker nodes (more easily seen on a larger cluster)

05. Experiment workflow



06. Implications

- Removed dependency on a centralised scheduler in times of fault tolerance
- Horizontal scaling - each added node brings extra gossip and increased scheduling capacity.
- Improved overall system health - replica gaps close before the control plane even notices a failure

07. Future work

- Implementation in Go.
- Implementation in smaller VM worker nodes <30 MB RAM to simulate real IoT context.
- Implementation in larger scale cluster.
- Stress test, how long can the WAN be out for?
- Validation vs KubeEdge "offline mode".
- Open source helm chart.

github.com/dlamagna/kubernetes-edge-healer